Birzeit University
Faculty of Engineering and Technology
Department of Electrical and Computer Engineering

ENCS5342 — Information Retrieval with Applications of NLP

Assignment 2: N-gram Language Models

Instructor: Ahmed I. A. Shawahna
Semester: T1252
Deadline: April 30, 2026
Total Points: 90

Overview

In this assignment, you will practice building and configuring n-gram language models (LMs) for multiple languages. You will compare different LM configurations — including tokenization techniques — and study how each configuration affects LM performance on in-domain and out-of-domain text.

Environment: Shell Scripts

All tasks in this assignment must be completed using Bash shell scripts running in a Unix/Linux environment. The provided helper script **task1.sh** demonstrates the required workflow for Task 1 and serves as a model for writing your own scripts for Tasks 2 and 3.

Windows Users — Windows Subsystem for Linux (WSL) Required

If you are working on a Windows computer, you are recommended to install Windows Subsystem for Linux (WSL) before beginning this assignment. WSL provides a full Linux environment inside Windows without requiring a separate virtual machine or dual-boot setup.

Installation steps:

1. Open PowerShell as Administrator and run: **wsl --install**
2. Restart your computer when prompted.
3. Launch the Ubuntu app from the Start menu and create a Unix username and password.
4. Update packages: **sudo apt update && sudo apt upgrade**

5. Install Perl: **sudo apt install perl**
6. Install Python: **sudo apt install python3 python3-pip**
7. Follow the *SRILM* and *subword-nmt* installation instructions from within the WSL terminal.

All shell commands in this assignment must be run inside the WSL terminal.
Do NOT use Windows Command Prompt or PowerShell to execute the assignment scripts.

WSL documentation: https://learn.microsoft.com/en-us/windows/wsl/install

Shell scripting resources:

- Bash scripting guide: `https://flaviocopes.com/bash-scripting/`
- GNU Bash manual: `https://www.gnu.org/software/bash/manual/`

Resources

All datasets and helper scripts are provided in the Assignment 2 zipped folder. You must also install the two tools described in Section B below before starting.

A. Contents of the Zipped Folder

Data Sets

- **English-Mix** subdirectory — English corpora from multiple genres:
  - `UNCorpus.train` United Nations Corpus **training** text
  - `UNCorpus.test` United Nations Corpus **test** text (in-domain)
  - `Bible.test` Bible Corpus **test** text (out-of-domain)
  - `Fair.test` My Fair Lady **test** transcript (out-of-domain)

- **Europarl** subdirectory — European Parliament data in 22 languages, organized in three sub-directories:
  - `train/train.*` Training files (one per language; two-letter language code extension)
  - `dev/dev.{1-22}` Development files for system tuning
  - `dev/dev.gold` Gold-standard language labels for the development set
  - `test/test.{1-15}` Blind test files — gold labels are not provided

The **Europarl** training files are organized by language family:

Language Family	Languages (ISO 639-1 codes)
Baltic	Lithuanian (lt), Latvian (lv)
Finno-Ugric	Hungarian (hu), Finnish (fi), Estonian (et)
Germanic	English (en), German (de), Danish (da), Dutch (nl), Swedish (sv)

Language Family	Languages (ISO 639-1 codes)
Greek	Greek (el)
Romance	Romanian (ro), Portuguese (pt), Italian (it), Spanish (es), French (fr)
Semitic	Maltese (mt)
Slavic	Slovak (sk), Czech (cs), Polish (pl), Bulgarian (bg), Slovene (sl)

Helper Scripts and Files

- Porter Stemmer is provided in the script **`porter.pl`**
- A simple tokenizer and detokenizer are provided in the scripts **`tokenizer.pl`** and **`detokenizer.pl`** (from the Moses Project) – They use the file **`nonbreaking_prefix.en`**.
- **`task1.sh`** Bash script that runs Task 1 commands — study this before writing your own scripts.

B. Tools to Install

SRILM Toolkit

- Download and install SRILM from: `http://www.speech.sri.com/projects/srilm/`
- Follow the instructions in the `INSTALL` file included in the distribution.
- SRILM is a powerful and versatile toolkit. Familiarize yourself with its manual pages at: `http://www.speech.sri.com/projects/srilm/manpages/`
- The primary commands you will use are ***`ngram-count`***, ***`ngram`***, and ***`disambig`***. Many of these commands accept more than 100 parameters. The `srilm-faq` manual page is a useful starting point.

BPE Tokeniser (subword-nmt)

- Install Rico Sennrich's **subword-nmt** from: `https://github.com/rsennrich/subword-nmt`
- Follow the installation instructions provided in the repository README.
- The two modes you will use are ***`learn_bpe`*** and ***`apply_bpe`***.
- Run ***`learn_bpe -h`*** or ***`apply_bpe -h`*** to see a full list of options.

Submission Instructions

Submission Format

Submit a single PDF file named:

Assignment2_<Student1_ID>_<Student2_ID>.pdf

For example, if your student IDs are 1230567 and 1230321, the file must be named:

Assignment2_1230567_1230321.pdf

Upload this PDF to the course page on Moodle/Blackboard before the stated deadline.

Your PDF must contain the following evidence for each question:

- Your written answer to the question.
- Screenshots of the terminal showing the exact commands you ran and their full output.
- For Python code (Task 3), the complete source code listing together with its output.
- For table-filling questions, the completed table with all values filled in.
- For questions that produce output files (e.g., `test.pred`), the file contents or a screenshot thereof.

Pair Work

This assignment is to be completed in pairs. Each pair should submit a single PDF, with both student IDs included in the file name as indicated above.

Task 1 — Let's Build a Language Model! (10 points)

This task walks you through building a basic language model step by step and evaluating it on in-domain and out-of-domain text. All commands must be run as a Bash shell script.

Step 1 — Tokenize the Text

Run the following commands from the root of the assignment folder. You may place them in a shell script.

```
cd Tools/
perl tokenizer.pl -no-escape < ../English-Mix/UNCorpus.train >
../English-Mix/UNCorpus.train.tok
perl tokenizer.pl -no-escape < ../English-Mix/UNCorpus.test  >
../English-Mix/UNCorpus.test.tok
perl tokenizer.pl -no-escape < ../English-Mix/Bible.test     >
../English-Mix/Bible.test.tok
cd ..
```

Question 1.a (1 point) What is the effect of this tokenization step? Describe the changes made to the text. (Hint: examine the file *nonbreaking_prefix.en*.)

Question 1.b (2 points) Compare the raw and tokenized files in terms of the number of tokens (total word occurrences) and the number of types (unique tokens). Explain the observed differences given the tokenisation that was applied.

Step 2 — Build the Language Model

Refer to the SRILM manual page for ngram-count for a description of all available parameters.

```
ngram-count -text English-Mix/UNCorpus.train.tok \
            -order 3 \
            -lm English-Mix/UNCorpus.train.tok.3.lm \
            -addsmooth 0.01
```

For the following three questions, refer to the SRILM manual page for ngram-format.

Question 1.c (1 point) What is the total number of 1-gram, 2-gram, and 3-gram entries in the resulting LM file?

Question 1.d (1 point) What is the purpose of each column in the 1-gram section of the LM file?

Question 1.e (1 point) Why does the 3-gram section have only two columns instead of three?

Step 3 — Evaluate Perplexity

Refer to the SRILM manual page for ngram for a description of all available parameters. Run the following commands and study the output carefully.

```
# In-domain test
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -order 3 -ppl English-
Mix/UNCorpus.test.tok
```

```
# Expected: file English-Mix/UNCorpus.test.tok: 500 sentences, 19169
words, 16 OOVs
# 0 zeroprobs, logprob= -25308.24 ppl= 19.39789 ppl1= 20.95907

# Out-of-domain test
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -order 3 -ppl English-
Mix/Bible.test.tok
# Expected: file English-Mix/Bible.test.tok: 500 sentences, 13999 words,
3053 OOVs
# 0 zeroprobs, logprob= -36285.73 ppl= 1479.676 ppl1= 2065.266
```

From the SRILM FAQ (with some paraphrasing)

By default any word not observed in the training data is considered OOV and OOV words are silently ignored by ***ngram*** *during perplexity (ppl) calculation. In the example above (with Bible.test.tok):*

```
file English-Mix/Bible.test.tok: 500 sentences, 13999 words, 3053 OOVs
0 zeroprobs, logprob= -36285.73 ppl= 1479.676 ppl1= 2065.266
```

There are ***3053*** *out-of-vocabulary words. This is the number of unknown word tokens, i.e. tokens that appear in* `Bible.test.tok` *but not in* `UNCorpus.train.tok` *from which* `UNCorpus.train.tok.3.lm` *was generated.*

logprob (log probability) *is* ***-36285.73.*** *This is the total logprob ignoring the 3053 unknown word tokens. The logprob does include the probabilities assigned to </s> tokens which are introduced by ngram-count. Thus, the total number of tokens which this logprob is based on is*

words - OOVs + sentences = 13999 - 3053 + 500

ppl (perplexity) is 1479.676. *This is the geometric average of 1/probability of each token, i.e., perplexity. The exact expression is:*

ppl = 10^(-logprob / (words - OOVs + sentences))

ppl1 *is the average perplexity per word excluding the </s> tokens.*

Question 1.f (1 point) Why is the perplexity of the UN Corpus LM different when evaluated on the Bible test set compared to the UN test set?

Question 1.g (1 point) Compute the OOV rate (100 × OOVs ÷ words) for both the UN and the Bible test files.

Question 1.h (1 point) Why is the OOV rate different for the Bible test set compared to the UN test set?

Step 4 — Generate Text

```
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -gen 1
# Expected: 62 increasing need for adequate of the Rome Statute World
Conference on Women and ratifying traditional knowledge and experience
mine-action participate in the field ,
```

Question 1.i (1 point) Provide the text generated by your run of the command above. Comment on its local fluency (does each word follow naturally from the previous one?) and its global coherence (does the overall sentence convey a meaningful idea?).

Task 2 — How Many Ways to Build a Language Model? (40 points)

In this task, you will systematically compare LM configurations across four independent variables. Vary one factor at a time while holding all others fixed. All experiments must be implemented as shell scripts.

Variables

1. Tokenization (applied to both training and test files)

- **Raw** — the original, untokenized text
- **Simple tokenization** — apply the Moses tokenizer from Task 1 (Give the files the extension: `.tok`)
- **Lowercased** — lowercase the simple-tokenized text (Give the files the extension: `.tok.lc`)
- **Porter stemmed** — apply the Porter stemmer to the lowercased text (Give the files the extension: `.tok.lc.port`)
- **BPE** — apply Byte Pair Encoding to the lowercased text (Give the files the extension: `.tok.lc.bpe`). **Important:** use default settings for minimum frequency and symbol count; learn the BPE merge rules on the training corpus only.

2. Training Size (applied to the training file only)

Setting	Lines
100%	70,000
50%	35,000
25%	17,500
12.5%	8,750
6.25%	4,375

3. LM Order

- 1 (unigram), 2 (bigram), 3 (trigram — default), 4 (4-gram), 5 (5-gram)

4. Smoothing Method

- Laplace (Add-1): `-addsmooth 1`
- Add-k (k = 0.1): `-addsmooth 0.1`
- Good-Turing (SRILM default — ignore any warning messages)
- Witten-Bell: `-wbdiscount`
- Kneser-Ney: `-kndiscount`

Evaluation metrics: You will evaluate the LMs produced using **OOV rate** and **perplexity**.

Test domains: You will compare performance of training data (UN Corpus) on one in-domain (UNCorpus.test) and two out-of-domain settings (Bible.test and Fair.test).

> **Scale of the Experiments**
> The four variables above define 5 × 5 × 5 × 5 = 625 possible LM configurations, producing up to 1,875 results (3 test domains × 625 configurations). In the experiments below, you vary one factor at a time, keeping all other factors fixed, to identify the effect of each variable individually.

Experiment 2.1 — Effect of Tokenization

Fixed: 100% training data, trigram order (3), Witten-Bell smoothing.

Apply all five tokenziations to both the training and the corresponding test files. Train each LM on the tokenized training data and evaluate it on the same tokenization of the test data.

Question 2.1.a (4 points) Run the experiments and fill in the table below.

Test file	Words	OOV	OOV %	PPL
`UNCorpus.test`				
`UNCorpus.test.tok`				
`UNCorpus.test.tok.lc`				
`UNCorpus.test.tok.lc.port`				
`UNCorpus.test.tok.lc.bpe`				
`Bible.test`				
`Bible.test.tok`				
`Bible.test.tok.lc`				
`Bible.test.tok.lc.port`				
`Bible.test.tok.lc.bpe`				
`Fair.test`				
`Fair.test.tok`				
`Fair.test.tok.lc`				
`Fair.test.tok.lc.port`				
`Fair.test.tok.lc.bpe`				

Question 2.1.b (2 points) What is the relationship between tokenization choice and OOV rate for in-domain versus out-of-domain test sets?

Question 2.1.c (2 points) What is the relationship between tokenization choice and perplexity?

Question 2.1.d (2 points) Based on the OOV rates and perplexity values, what do the results suggest about the similarity between the Bible and the UN Corpus, versus My Fair Lady and the UN Corpus?

Experiment 2.2 — Effect of Training Size

Fixed: lowercased tokenization (**tok.lc**), trigram order (3), Witten-Bell smoothing.

Train five separate LMs using progressively smaller subsets of the training data. Evaluate each LM on all three test domains.

Question 2.2.a (4 points) Run the experiments and fill in the table below.

Test file	Train lines	Words	OOV	OOV %	PPL
UNCorpus.test	70,000				
UNCorpus.test	35,000				
UNCorpus.test	17,500				
UNCorpus.test	8,750				
UNCorpus.test	4,375				
Bible.test	70,000				
Bible.test	35,000				
Bible.test	17,500				
Bible.test	8,750				
Bible.test	4,375				
Fair.test	70,000				
Fair.test	35,000				
Fair.test	17,500				
Fair.test	8,750				
Fair.test	4,375				

Question 2.2.b (2 points) What is the relationship between training size and OOV rate?

Question 2.2.c (2 points) What is the relationship between training size and perplexity for in domain data?

Question 2.2.d (2 points) What is the relationship between training size and perplexity for out-of-domain data? Is the pattern similar to or different from the in-domain case? Explain.

Experiment 2.3 — Effect of LM Order

Fixed: lowercased tokenization (**tok.lc**), 100% training data, Witten-Bell smoothing.

Train five LMs of orders 1 through 5. Make sure to train and evaluate on the same tokenization and in the same order (use ***-order*** in the ***ngram*** command to override the default 3).

Question 2.3.a (4 points) Run the experiments and fill in the table below.

Test file	Order	Words	OOV	OOV %	PPL
`UNCorpus.test`	1				
`UNCorpus.test`	2				
`UNCorpus.test`	3				
`UNCorpus.test`	4				
`UNCorpus.test`	5				
`Bible.test`	1				
`Bible.test`	2				
`Bible.test`	3				
`Bible.test`	4				
`Bible.test`	5				
`Fair.test`	1				
`Fair.test`	2				
`Fair.test`	3				
`Fair.test`	4				
`Fair.test`	5				

Question 2.3.b (2 points) What is the relationship between LM order and OOV rate?

Question 2.3.c (2 points) What is the relationship between LM order and perplexity for in domain data?

Question 2.3.d (2 points) What is the relationship between LM order and perplexity for out-of-domain data? Is the pattern similar to or different from the in-domain case? Explain.

Experiment 2.4 — Effect of Smoothing Method

Fixed: lowercased tokenization (**tok.lc**), 100% training data, 5-gram order.

Build five LMs using each of the five smoothing methods. Make sure to train and evaluate on the same tokenization and in the same order (use ***-order*** in the ***ngram*** command to override the default 3).

Question 2.4.a (4 points) Run the experiments and fill in the table below.

Test file	Smoothing	Words	OOV	OOV %	PPL
UNCorpus.test	Add-1				
UNCorpus.test	Add-0.1				
UNCorpus.test	Good-Turing				
UNCorpus.test	Witten-Bell				
UNCorpus.test	Kneser-Ney				
Bible.test	Add-1				
Bible.test	Add-0.1				
Bible.test	Good-Turing				
Bible.test	Witten-Bell				
Bible.test	Kneser-Ney				
Fair.test	Add-1				
Fair.test	Add-0.1				
Fair.test	Good-Turing				
Fair.test	Witten-Bell				
Fair.test	Kneser-Ney				

Question 2.4.b (2 points) What is the relationship between the smoothing method and perplexity for in domain data?

Question 2.4.c (2 points) What is the relationship between the smoothing method and perplexity for out-of-domain data? Is the pattern similar to or different from the in-domain case? Explain.

Question 2.4.d (2 points) Which smoothing method performs best overall? Justify your answer with reference to the results.

Task 3 — Guess the Language! (40 points)

Using the **Europarl/train** files, build multiple language models and use them to identify the language of the files in the **Europarl/dev** files. The correct labels for the **Europarl/dev** are in **Europarl/dev.gold**. A common method for doing this task is to use the perplexity of a text against all language models, and select the language of the model with lowest perplexity.

> **Key Hint**
> Build your language models at the character level (each character, including spaces, is a token), not at the word level. Character-level models generalize better across languages.

Training data: `Europarl/train/train.*`

Development data: `Europarl/dev/dev.{1-22}` Gold labels: `Europarl/dev/dev.gold`

Test data: `Europarl/test/test.{1-15}` (gold labels are not provided)

Question 3.1 (20 points)

Write a Python script named identify-language.py that operates in three modes:

```
python identify-language.py TRAIN <train-dir>/<train-base> <model-dir>
python identify-language.py PREDICT <test-dir> <model-dir>
python identify-language.py EVALUATE <gold-file> <prediction-file>
```

The script must meet the following requirements:

1. Recognize all 22 language labels: (`it`, `lt`, `et`, `fr`, `hu`, `lv`, `cs`, `en`, `da`, `de`, `mt`, `nl`, `pl`, `fi`, `pt`, `ro`, `sk`, `sl`, `sv`, `bg`, `el`, `es`).
2. Training files follow the naming convention `<train-base>.<label>`, where `<label>` is the two-character ISO 639-1 language code.
3. Test files have a numeric extension (e.g., `dev.12`, `test.3`).
4. PREDICT mode must write its predictions to standard output, one language code per line.
5. Your submission must include the complete Python source code and the accuracy achieved on the development set.

To call SRILM from within Python, use **os.system()**:

```
import os
cmd = "ls"
os.system(cmd)

# Expected: Applications     Movies
            Desktop          Music
            Documents        Pictures
            Downloads        Public
            Dropbox          Untitled
            Library
```

To use perl regex with a number of languages other than English, you will need to use the following instruction to work with unicode:

```
perl -pe 'use open \":std\",\":encoding(UTF-8)\"; <YOUR REGEX>;'
```

The three basic ways you will use the code are the following:

1) To train on the train data and create a model directory that houses all the .lm files

```
python identify-language.py TRAIN Europarl/train/train modeldir
```

2) To predict the answers for the dev set and print them to STDIN/file

```
python identify-language.py PREDICT Europarl/dev modeldir > dev.predict
```

3) To evaluate the predictions against the gold

```
python identify-language.py EVALUATE Europarl/dev/dev.gold dev.predict
```

When trained on the full **Europarl** training data and evaluated on the complete development files, your system is expected to achieve 100% accuracy on the development set.

Question 3.2 (5 points)

Modify your script to train with one random line only from training, and to use one random line only from the development files, and LM order 1.

- (1 point) Run your modified system and report the accuracy on the development set.
- (4 points) It is reasonable to expect the accuracy to go down. Is there a pattern for the errors?

Question 3.3 (15 points)

Using your best-performing model configuration from Questions 3.a and 3.b, predict the language of each file in the blind test set (**Europarl/test/test.{1-15}**).

Submit your predictions in a file named **test.pred**. The file must contain two tab-separated columns — the test file name and the predicted language code — as shown below:

```
test.1        <lang>
test.2        <lang>
...
test.15       <lang>
```

The format must match that of **Europarl/dev/dev.gold**.

Grading Summary

Task	Description	Points
Task 1	Building and evaluating a basic language model	10
Task 2.1	Effect of tokenization	10
Task 2.2	Effect of training size	10
Task 2.3	Effect of LM order	10
Task 2.4	Effect of smoothing method	10
Task 3.1	Language identification system	20
Task 3.2	Analysis with reduced training and evaluation	5
Task 3.3	Blind test-set language prediction	15
	Total	**90**

Academic Integrity

All submitted work must be your own. You may discuss high-level approaches with classmates, but all code, scripts, and written answers must be written independently.
Any form of plagiarism will result in a grade of zero for the assignment and may be subject to further disciplinary action in accordance with university policy.